

PROGRAMACIÓN II

INGENIERÍA DE EN DESARROLLO DE SOFTWARE Y

CIENCIA DE DATOS

SISTEMA DE GESTIÓN ACADÉMICO Y ADMINISTRATIVO

ENTREGA 2

EDGARD SANTIAGO GRANADOS MARTÍNEZ	CIF 2026010384
ROBERTO CARLOS GUTIERREZ PORTILLO	CIF 2026010156
JORGE MANUEL MELENDEZ CARPIO	CIF 2026010487
WALTER ERNESTO VÁSQUEZ MELARA	CIF 2026010090
JOSUE ALEXANDER ERAZO HERNANDEZ	CIF 2026010109
AMILCAR ALEXANDER CRESPIN HERNANDEZ	CIF 2026011225
BRYAN ALEXANDER REYES ALVARADO	CIF 2026011706
FRANSHESKA JERUSALÉN BENITEZ MUÑOZ	CIF 2026010787
HECTOR GERARDO GONZALEZ MORANA	CIF 2026010070
JOSÉ LUIS IRAHETA MARROQUÍN	CIF 2026010246
EDENILSON ALEXANDER AMAYA BENITEZ	CIF 2026010437

27 de Septiembre de 2026

Contenido

OBJETIVO GENERAL 3
OBJETIVOS ESPECÍFICOS 3
DISTRIBUCIÓN DEL EQUIPO SCRUM 4
DEFINICIÓN DE ROLES Y FUNCIONES POR ROL DEL SISTEMA 5
REQUERIMIENTOS DEL SISTEMA 6
ALCANCES Y LIMITACIONES 10
PLANIFICACIÓN Y CRONOGRAMA 11
ENTRADAS Y SALIDAS DEL SISTEMA 12
DECLARACIÓN DE IDENTIDADES 14
EVOLUCIÓN PARA ENTREGA 2 16
Estructura actual del proyecto 16
Módulos identificados 16
ARQUITECTURA POR CAPAS 17
PROGRAMACIÓN ORIENTADA A OBJETOS 17
Encapsulamiento 17
Abstracción mediante interfaces 18
Herencia y polimorfismo 18
Clases abstractas 18
COLECCIONES Y ESTRUCTURAS DE DATOS 19
PERSISTENCIA MEDIANTE ARCHIVOS .DAT 19
Flujo requerido para completar la persistencia 20
CONCURRENCIA 20
Caso de uso propuesto 20
PRUEBAS Y CALIDAD 21
INTEGRACIÓN Y ESTADO FUNCIONAL 21
CONTROL DE VERSIONES Y GIT 23
CONCLUSIONES 25
BIBLIOGRAFÍA 25
ANEXO A. REFERENCIAS TÉCNICAS DEL REPOSITORIO 26

OBJETIVO GENERAL

Desarrollar un sistema integrado de gestión académica y administrativa para una academia de formación continua, utilizando Java y una base de datos, que permita gestionar de manera eficiente y organizada la información, así como también los principales procesos académicos y administrativos de la institución.

OBJETIVOS ESPECÍFICOS

1. Diseñar la estructura del sistema mediante la definición de los roles, requerimientos, funcionalidades y entidades necesarias para representar los procesos académicos y administrativos de la academia.

2. Evolucionar el diseño de la primera entrega hacia una implementación orientada a objetos en Java, aplicando encapsulamiento, herencia, polimorfismo, clases abstractas y colecciones de forma coherente con el dominio.

3. Organizar el código mediante una arquitectura por capas que separe presentación/controladores, lógica de negocio, repositorios y persistencia.

4. Implementar persistencia local mediante archivos .dat para mantener información entre ejecuciones, dejando la conexión a PostgreSQL para una etapa posterior.

5. Incorporar pruebas automatizadas y evidencia de integración mediante Git y GitHub, manteniendo trazabilidad de cambios por funcionalidades y ramas.

DISTRIBUCIÓN DEL EQUIPO SCRUM

N°	Nombre completo	Rol Scrum	Rol técnico
1	Edgard Santiago Granados Martínez	Product Owner (Business Owner)	Analista
2	Roberto Carlos Gutiérrez Portillo	QA	Analista de Pruebas
	Héctor Gerardo González Moran	QA	Analista de Pruebas
3	Walter Ernesto Vasquez Melara	Developer	Desarrollador Backend (Java)
4	Jorge Manuel Melendez Carpio	Developer	Desarrollador Backend (Java)
5	José Luis Iraheta Marroquín	Developer	Desarrollador Backend (Java)
6	Edenilson Alexander Amaya Benitez	Developer	Desarrollador Backend (Java)
7	Fransheska Jerusalén Benitez Muñoz	Developer	Desarrollador Backend (Java)
8	Bryan Alexander Reyes Alvarado	Developer	Desarrollador Backend (Java)
9	Amilcar Alexander Crespin Hernández	Developer	Desarrollador Backend (Java)
10	Josué Alexander Hernández Erazo	Developer	Desarrollador Backend (Java)
11	Jorge Alberto Ramos Cabañas	Developer	Desarrollador Backend (Java)

DEFINICIÓN DE ROLES Y FUNCIONES POR ROL DEL SISTEMA

Rol	Descripción	Funciones del Sistema
Administrador del Sistema	Es el responsable de administrar la plataforma, controlar el acceso de los usuarios y garantizar el correcto funcionamiento del sistema de gestión académica y administrativa.	• Iniciar sesión. • Registrar nuevos usuarios. • Asignar roles a los usuarios. • Modificar información de usuarios. • Activar y desactivar usuarios. • Garantizar el acceso según el rol asignado. • Administrar las credenciales de acceso.
Coordinador Académico	Es el encargado de planificar y supervisar la oferta académica de la academia, así como la asignación de instructores y el seguimiento del rendimiento de los cursos.	• Registrar cursos. • Administrar la oferta académica. • Asignar instructores a los cursos. • Consultar el rendimiento académico por curso. • Supervisar el desempeño académico de los estudiantes. • Gestionar los períodos académicos.
Instructor	Es el responsable de impartir los cursos, proporcionar material didáctico y evaluar el desempeño académico de los estudiantes.	• Consultar los estudiantes matriculados. • Publicar material didáctico. • Crear actividades evaluativas. • Calificar actividades entregadas. • Registrar observaciones sobre las evaluaciones. • Consultar la información de los cursos asignados.
Estudiante	Es la persona inscrita en la academia que utiliza el sistema para gestionar su proceso de formación académica y consultar su información.	• Matricularse en cursos. • Registrar pagos. • Consultar y descargar material didáctico. • Consultar actividades asignadas. • Entregar actividades. • Consultar calificaciones. • Consultar su historial académico.
Asesor Comercial	Es el responsable de atender a los estudiantes en los procesos financieros relacionados con pagos y estado de cuenta.	• Validar pagos registrados por los estudiantes. • Aprobar o rechazar pagos. • Actualizar el estado de cuenta. • Consultar el estado de cuenta de los estudiantes. • Verificar la existencia de mora.

		• Brindar atención sobre consultas de pagos.
Coordinador Comercial	Es el encargado de supervisar la gestión de cobranza y analizar los indicadores financieros relacionados con la mora de los estudiantes.	• Consultar el listado de estudiantes en mora. • Consultar indicadores de mora. • Filtrar información por período académico. • Filtrar información por programa o formación. • Supervisar la gestión de cobranza
Asistente Administrativo	Es el responsable de gestionar los procesos administrativos relacionados con la facturación y la atención administrativa de los estudiantes.	• Generar facturas a partir de pagos validados. • Consultar el historial de facturación de los estudiantes. • Consultar el detalle de las facturas emitidas. • Brindar atención administrativa relacionada con la facturación

REQUERIMIENTOS DEL SISTEMA

Prioridad	Historia de Usuario	Criterios de aceptación
HU-001	Como **usuario del sistema** quiero iniciar sesión para acceder únicamente a las funcionalidades correspondientes a mi rol.	• Debe permitir ingresar usuario y contraseña. • Debe validar las credenciales. • Debe identificar el rol del usuario. • Debe impedir el acceso con credenciales incorrectas. • Debe mostrar el menú correspondiente al rol autenticado.
HU-002	Como **coordinador académico** quiero registrar cursos nuevos para ofertarlos durante un período académico.	• Debe registrar nombre, código y descripción del curso. • Debe asociar un período académico.
HU-003	Como **coordinador académico** quiero asignar un instructor a un curso para definir quién impartirá la formación.	• Debe permitir seleccionar un curso. • Debe permitir seleccionar un instructor. • Debe registrar la asignación. • Debe cambiar el estatus a “activo” al asignarse el instructor
HU-004	Como **coordinador académico** quiero registrar períodos académicos para organizar la oferta de cursos en cada ciclo de formación.	• Debe permitir registrar el nombre del período académico. • Debe permitir registrar la fecha de inicio y la fecha de finalización. • Debe validar que las fechas sean válidas. • Debe impedir registrar períodos con nombres duplicados.

		• Debe permitir cambiar el estado del período (Activo, Finalizado o Planificado). • El período registrado debe estar disponible para asociarlo a los cursos.
HU-005	Como **estudiante** quiero matricularme en un curso para formar parte del grupo académico.	• Debe permitir seleccionar un curso. • Debe registrar la matrícula correctamente.
HU-006	Como **estudiante** quiero registrar un pago realizado para informar a la academia que he efectuado el pago correspondiente.	• Debe permitir ingresar el monto pagado. • Debe registrar la fecha del pago. • Debe permitir adjuntar un comprobante. • El pago debe quedar en estado **Pendiente de validación**.
HU-007	Como **asesor comercial** quiero validar los pagos registrados por los estudiantes para actualizar correctamente su estado de cuenta.	• Debe mostrar únicamente pagos pendientes. • Debe visualizar el comprobante. • Debe aprobar o rechazar el pago. • Debe actualizar automáticamente el estado de cuenta. • Debe registrar quién realizó la validación.
HU-008	Como **asistente administrativo** quiero generar una factura a partir de un pago validado para documentar la transacción.	• Debe permitir seleccionar un pago validado. • Debe generar un número de factura. • Debe asociar la factura al estudiante. • Debe almacenar la factura en el sistema.
HU-009	Como **instructor** quiero consultar los estudiantes matriculados en mis cursos para administrar correctamente mis clases.	• Debe mostrar únicamente estudiantes del curso seleccionado. • Debe permitir buscar estudiantes. • Debe mostrar nombre del estudiante.
HU-010	Como **instructor** quiero publicar material didáctico para apoyar el aprendizaje de mis estudiantes.	• Debe permitir seleccionar un curso. • Debe permitir cargar archivos. • Debe asociar el material al curso. • Debe estar disponible únicamente para estudiantes matriculados.
HU-011	Como **instructor** quiero crear actividades evaluativas para que los estudiantes puedan entregar sus trabajos.	• Debe permitir seleccionar un curso. • Debe registrar nombre y descripción de la actividad. • Debe registrar la fecha límite. • Debe definir la ponderación de la actividad. • Debe publicar la actividad para los estudiantes.
HU-012	Como **estudiante** quiero consultar y descargar el material del curso para apoyar mi proceso de aprendizaje.	• Debe mostrar el listado de materiales publicados. • Debe permitir descargar archivos.

		• Debe restringir el acceso a estudiantes matriculados.
HU-013	Como **estudiante** quiero consultar las actividades asignadas para conocer las tareas pendientes.	• Debe mostrar únicamente actividades del estudiante. • Debe mostrar fecha límite. • Debe mostrar descripción y ponderación. • Debe indicar si la actividad ya fue entregada.
HU-014	Como **estudiante** quiero entregar una actividad para que pueda ser evaluada por el instructor.	• Debe permitir seleccionar la actividad. • Debe permitir adjuntar un archivo. • Debe registrar la fecha de entrega. • Debe asociar la entrega al estudiante y a la actividad.
HU-015	Como **instructor** quiero calificar las actividades entregadas para evaluar el desempeño de los estudiantes.	• Debe mostrar las entregas recibidas. • Debe permitir registrar una calificación. • Debe permitir agregar observaciones. • Debe almacenar la evaluación.
HU-016	Como **estudiante** quiero consultar mis calificaciones para conocer mi rendimiento académico.	• Debe mostrar únicamente las notas del estudiante autenticado. • Debe mostrar actividad, calificación y observaciones. • Debe calcular el promedio del curso.
HU-017	Como **estudiante** quiero consultar mi historial académico para conocer los cursos finalizados y sus resultados.	• Debe mostrar únicamente cursos finalizados. • Debe mostrar nota final.
HU-018	Como **asesor comercial** quiero consultar el estado de cuenta de un estudiante para brindar atención y verificar si posee mora.	• Debe permitir buscar estudiantes. • Debe mostrar pagos registrados. • Debe mostrar saldo pendiente. • Debe indicar si existe mora.
HU-019	Como **coordinador comercial** quiero consultar el listado de estudiantes en mora para gestionar la cobranza.	• Debe listar únicamente estudiantes con saldo pendiente. • Debe mostrar el monto adeudado. • Debe permitir ordenar por deuda. • Debe permitir buscar estudiantes específicos. • Debe de mostrar el curso al cual pertenece el estudiante en mora
HU-020	Como **coordinador comercial** quiero consultar indicadores de mora para presentar información a Dirección General.	• Debe mostrar cantidad de estudiantes en mora. • Debe mostrar monto total pendiente. • Debe permitir filtrar por período académico. • Debe permitir filtrar por formación.

HU-021	Como **coordinador académico** quiero consultar el rendimiento académico de cada curso para evaluar su desempeño.	• Debe mostrar promedio del curso. • Debe mostrar cantidad de aprobados y reprobados. • Debe permitir seleccionar el curso.
HU-022	Como **asistente administrativo** quiero consultar el historial de facturación de un estudiante para responder consultas administrativas.	• Debe permitir buscar un estudiante. • Debe mostrar todas las facturas emitidas. • Debe mostrar fecha y monto de cada factura. • Debe permitir consultar el detalle de una factura.
HU-023	Como **administrador del sistema** quiero registrar y administrar usuarios para controlar el acceso al sistema según el rol asignado.	• Debe permitir registrar nuevos usuarios • Debe permitir asignar un rol (Estudiante, Instructor, Coordinador Académico, Asesor Comercial, Coordinador Comercial o Asistente Administrativo). • Debe permitir modificar la información de un usuario. • Debe permitir activar o desactivar usuarios. • Debe impedir la creación de usuarios con nombres de usuario duplicados. • Debe almacenar de forma segura la contraseña del usuario.

ALCANCES Y LIMITACIONES

Alcances:

El sistema de gestión académica y administrativa estará orientado para apoyar los procesos principales de una academia de formación continua, permitiendo centralizar la información académica, administrativa y financiera en una sola plataforma.

Dentro del alcance del proyecto se encuentra:

- Gestionar el acceso al sistema mediante autenticación de usuarios y control de permisos según el rol asignado (Administrador, Coordinador Académico, Instructor, Estudiante, Asesor Comercial, Coordinador Comercial y Asistente Administrativo).
- Administrar la oferta académica mediante el registro de cursos, períodos académicos y la asignación de instructores.
- Permitir que los estudiantes realicen la matrícula en los cursos disponibles y consulten su historial académico.
- Facilitar la publicación y descarga de material didáctico, así como la creación, entrega y evaluación de actividades académicas.
- Gestionar el registro, validación y seguimiento de pagos realizados por los estudiantes.
- Generar facturas a partir de los pagos previamente validados.
- Proporcionar consultas e indicadores relacionados con el rendimiento académico y el estado de pago de cuotas de los estudiantes, incluyendo reportes de mora.
- Centralizar la información de los procesos académicos y administrativos para mejorar el control y la organización de la academia.

Limitaciones:

El sistema no incluirá las siguientes funcionalidades:

- No diferenciará distintos tipos de pago según montos o categorías.
- No manejará diferentes métodos de pago, como tarjetas de crédito, transferencias bancarias o billeteras electrónicas.
- No contará con una pasarela de pagos integrada para realizar transacciones en línea.
- No enviará notificaciones automáticas relacionadas con pagos, tareas, fechas importantes o avisos académicos.
- No generará certificados o constancias de estudio.
- No incluirá procesos de autogestión adicionales que excedan las funcionalidades definidas en los requerimientos del proyecto.
- La interfaz de usuario (UX/UI) no será un objetivo principal del desarrollo, por lo que será más importante la funcionalidad sobre el diseño visual

PLANIFICACIÓN Y CRONOGRAMA

Fases del Proyecto

- **Fase 1: Inicio y Requerimientos (Semanas 1 y 2):** Análisis de HU-001 a HU-023, diseño de arquitectura del sistema e infraestructura de BD.
- **Fase 2: Sprint 1 - Módulo Base y Usuarios (Semanas 3 y 4):** Autenticación y gestión de usuarios (HU-001, HU-023).
- **Fase 3: Sprint 2 - Gestión Académica (Semanas 5 y 6):** Períodos, cursos, matrículas y aulas virtuales (HU-002 a HU-005, HU-009 a HU-017).
- **Fase 4: Sprint 3 - Gestión Administrativa y Pagos (Semanas 7 y 8):** Pagos, validación comercial y facturación (HU-006 a HU-008, HU-018 a HU-020, HU-022).
- **Fase 5: Sprint 4 - Reportes y Métricas (Semanas 9 y 10):** Indicadores académicos y financieros (HU-021) y ajustes finales.
- **Fase 6: Cierre, Despliegue y Entrega (Semanas 11 y 12):** Pruebas integrales de QA, corrección de bugs y entrega final.

Código	Descripción de la Actividad	Integrantes / Responsables Principales
T-01	Definición de Requerimientos y Refinamiento del Backlog	Edgard Santiago Granados Martinez (Product Owner), Roberto Carlos Gutiérrez Portillo (QA), Hector Gerardo Gonzalez Moran (QA)
T-02	Diseño de Arquitectura Java y Base de Datos	Jorge Alberto Ramos Cabañas (Arquitecto), Edenilson Alexander Amaya Benitez (Base de Datos), Fransheska Jerusalen Benitez Muñoz (Base de Datos), Bryan Alexander Reyes Alvarado (Base de Datos)
T-03	Desarrollo de Autenticación y Usuarios (HU-001, HU-023)	Walter Ernesto Vasquez Melara (Dev Backend), José Luis Iraheta Marroquín (Dev Backend/Frontend)
T-04	Desarrollo del Módulo Académico (HU-002 a HU-005, HU-009 a HU-017)	Walter Ernesto Vasquez Melara (Dev Backend), Hector Gerardo Gonzalez Moran (Dev Backend), Jorge Manuel Melendez Carpio (Dev Backend), José Luis Iraheta Marroquín (Dev Backend/Frontend)
T-05	Desarrollo del Módulo Financiero (HU-006 a HU-008, HU-018 a HU-020, HU-022)	Jorge Manuel Melendez Carpio (Dev Backend), José Luis Iraheta Marroquín (Dev Backend/Frontend), Hector Gerardo Gonzalez Moran (Dev

		Backend)
T-06	Desarrollo de Métricas e Indicadores (HU-021)	Walter Ernesto Vasquez Melara (Dev Backend), Jorge Manuel Melendez Carpio (Dev Backend)
T-07	Plan de Pruebas, Control de Calidad (QA) y Criterios de Aceptación	Amilcar Alexander Crespin Hernandez (QA), Josue Alexander Hernandez Erazo (QA)
T-08	Integración Final, Corrección de Defectos y Cierre	Todo el Equipo Scrum

En el siguiente enlace se puede encontrar el cronograma del proyecto en Diagrama de Gantt:

Planificacion y Cronograma del proyecto

ENTRADAS Y SALIDAS DEL SISTEMA

Historia de Usuario	Entrada	Salida
HU-001 Iniciar sesión	Usuario, contraseña	Acceso al sistema o mensaje de error
HU-002 Registrar curso	Código, nombre, descripción, período	Curso registrado
HU-003 Asignar instructor	Curso, instructor	Instructor asignado
HU-004 Registrar período	Nombre, fecha inicio, fecha fin	Período registrado
HU-005 Matrícula	Estudiante, curso	Matrícula registrada
HU-006 Registrar pago	Monto, fecha, comprobante	Pago pendiente de validación
HU-007 Validar pago	Pago pendiente, decisión	Pago aprobado o rechazado
HU-008 Generar factura	Pago validado	Factura generada
HU-009 Consultar estudiantes	Curso	Lista de estudiantes

HU-010 Publicar material	Curso, archivo	Material publicado
HU-011 Crear actividad	Curso, nombre, descripción, fecha, ponderación	Actividad creada
HU-012 Consultar material	Curso	Material disponible
HU-013 Consultar actividades	Estudiante	Lista de actividades
HU-014 Entregar actividad	Actividad, archivo	Entrega registrada
HU-015 Calificar actividad	Entrega, nota, observaciones	Calificación registrada
HU-016 Consultar calificaciones	Estudiante	Calificaciones y promedio
HU-017 Historial académico	Estudiante	Cursos finalizados y notas
HU-018 Estado de cuenta	Estudiante	Saldo y mora
HU-019 Estudiantes en mora	Filtro de búsqueda	Lista de estudiantes en mora
HU-020 Indicadores de mora	Período, formación	Reporte de mora
HU-021 Rendimiento académico	Curso	Promedio, aprobados y reprobados
HU-022 Historial de facturación	Estudiante	Facturas emitidas
HU-023 Administrar usuarios	Datos del usuario	Usuario registrado o actualizado

DECLARACIÓN DE IDENTIDADES

1. Entidad: Usuario.

Esta entidad representa a cualquier persona que interactúa con el sistema, básicamente es la clave para la HU-001. Centraliza la información para la autenticación y el control de acceso a la plataforma, garantizando que cada usuario ingrese únicamente a las opciones autorizadas.

Atributos: ID_Usuario, Nombre_Usuario, Contraseña, Estado (Activo/Inactivo), ID_Rol.

2. Entidad: Rol.

Esta entidad está destinada a la definición de los perfiles de usuario dentro del sistema. Permite categorizar los accesos y restringir las funcionalidades de la interfaz según el nivel de responsabilidad asignado a cada cuenta.

Atributos: ID_Rol, Nombre_Rol, Descripción.

3. Entidad: Período Académico.

Esta es la entidad que representa la estructura temporal en los que se dividen las actividades académicas esto permite organizar la oferta académica en rangos de fechas específicos.

Atributos: ID_Periodo, Nombre_Periodo, Fecha_Inicio, Fecha_Fin, Estado (Activo, Planificado, Finalizado).

4. Entidad: Curso.

Esta entidad contiene la información de las materias ofertadas en un período académico específico, vinculando las actividades docentes a un instructor designado.

Atributos: ID_Curso, Código_Curso, Nombre_Curso, Descripción, ID_Periodo, ID_Instructor.

5. Entidad: Matrícula.

Esta es la entidad que registra la vinculación formal de un estudiante con un curso determinado. Funciona como tabla que valida que el usuario forma parte del grupo académico.

Atributos: ID_Matricula, ID_Estudiante, ID_Curso, Fecha_Matricula.

6. Entidad: Pago.

Esta entidad almacena el historial de las transacciones financieras enviadas por los estudiantes para notificar la cancelación de sus cuotas, y la registra de manera general sin clasificar modalidades de pago ni detallar variaciones de montos.

Atributos: ID_Pago, ID_Estudiante, Monto, Fecha_Pago, Ruta_Comprobante, Estado_Pago (Pendiente, Aprobado, Rechazado), ID_Asesor_Validador

7. Entidad: Factura.

Esta entidad es la encargada de tener un respaldo legal y administra los pagos que previamente han sido verificados y aprobados por el área comercial.

Atributos: ID_Factura, Numero_Factura, ID_Pago, ID_Estudiante, Fecha_Emisión, Monto_Total.

8. Entidad: Material Didáctico.

Esta entidad gestiona los recursos digitales, archivos y guías cargados al sistema para complementar el proceso de enseñanza dentro de un curso.

Atributos: ID_Material, ID_Curso, Título_Material, Ruta_Archivo, Fecha_Subida.

9. Entidad: Actividad Evaluativa

Esta entidad registra las tareas o exámenes programados dentro de un curso para medir el avance y rendimiento académico.

Atributos: ID_Actividad, ID_Curso, Nombre_Actividad, Descripción, Fecha_Límite, Ponderación.

10. Entidad: Entrega de Evaluación

Esta entidad almacena los archivos y respuestas enviadas por los alumnos para dar cumplimiento a una actividad evaluativa, sirviendo también como registro para la nota asignada y la retroalimentación.

Atributos: ID_Entrega, ID_Actividad, ID_Estudiante, Ruta_Archivo_Entrega, Fecha_Entrega, Calificación, Observaciones.

EVOLUCIÓN PARA ENTREGA 2

La rama main revisada presenta una evolución respecto del diseño inicial: el proyecto ya contiene paquetes de controller, service, repository, persistence, entity, enums, exception y pruebas. La solución dejó de ser únicamente una especificación de entidades y pasó a incorporar reglas de negocio, contratos de repositorio, servicios y pruebas automatizadas.

Estructura actual del proyecto

```
src/main/java/sv/gestionacademica/
├── controller
├── entity
├── enums
├── exception
├── persistence
├── repository
└── service
src/test/java/sv/gestionacademica/service/
```

Módulos identificados

Módulo	Clases/elementos localizados
Usuarios/autenticación	Usuario, Rol, Sesion, UsuarioService, AutenticacionService, UsuarioController, LoginController
Académico	PeriodoAcademico, Curso, repositories, services, AcademicoController
Matrícula	Matricula, MatriculaService
Financiero	Pago, PagoService, PagoRepository, FinancieroController
Evaluación	EntregaEvaluacion, EntregaRepository, EvaluacionService
Reportes	EstadoCuenta, IndicadorMora, EstadoCuentaService, ReporteService
Persistencia	DataFileManager, PersistenciaException
Pruebas	EstadoCuentaServiceTest, EvaluacionServiceTest, PagoServiceTest, ReporteServiceTest, UsuarioServiceTest

ARQUITECTURA POR CAPAS

La organización actual separa responsabilidades mediante controladores, servicios, repositorios y entidades. Para el avance, el término Repository se utiliza como la abstracción de acceso a datos; la capa persistence contiene el mecanismo de almacenamiento local. No se debe presentar la conexión a PostgreSQL como funcionalidad implementada en esta entrega.

Capa	Responsabilidad	Ejemplos
Controller	Recibir operaciones de la interfaz y delegarlas a servicios.	LoginController, UsuarioController, AcademicoController, FinancieroController.
Service	Aplicar reglas de negocio y coordinar operaciones.	UsuarioService, PeriodoAcademicoService, CursoService, PagoService, EvaluacionService.
Repository	Definir o encapsular operaciones de acceso a datos.	UsuarioRepository, CursoRepository, PeriodoAcademicoRepository, EntregaRepository, PagoRepository.
Persistence	Gestionar serialización y archivos.	DataFileManager, PersistenciaException.
Entity/Model	Representar el dominio.	Usuario, Curso, PeriodoAcademico, Matricula, Pago, EntregaEvaluacion.
Enums	Representar estados controlados.	EstadoUsuario, EstadoCurso, EstadoMatricula, EstadoPago, EstadoPeriodo.

La arquitectura permite que el mecanismo de persistencia cambie posteriormente de archivos .dat a PostgreSQL sin trasladar las reglas de negocio al controlador.

Controller → Service → Repository → Persistence → archivo .dat

↓

Entity / reglas de negocio

PROGRAMACIÓN ORIENTADA A OBJETOS

Encapsulamiento

El código actual utiliza atributos privados en las entidades y expone operaciones mediante métodos públicos. Por ejemplo, Usuario encapsula sus datos y ofrece getters/setters; PeriodoAcademico y Curso además concentran reglas de negocio en métodos específicos. Por ejemplo:

```
private String nombreUsuario;
private EstadoUsuario estado;
private Rol rol;

public EstadoUsuario getEstado() { return estado; }
public void setEstado(EstadoUsuario estado) { this.estado = estado; }
```

Abstracción mediante interfaces

UsuarioService y AutenticacionService están definidos como interfaces. Esto permite establecer contratos de servicio y facilita sustituir implementaciones. Sin embargo, en el estado actual revisado no se localizó una implementación productiva concreta de UsuarioServiceImpl o AutenticacionServiceImpl; por tanto, la existencia de la interfaz no debe presentarse por sí sola como evidencia de polimorfismo completo en producción.

Herencia y polimorfismo

En la implementación actual se optó por no establecer una jerarquía de herencia entre las entidades principales del dominio, debido a que las relaciones identificadas durante el análisis corresponden principalmente a asociaciones entre conceptos del sistema y no a relaciones de especialización del tipo “es-un”. Por ejemplo, un Usuario posee un Rol, una Matricula relaciona un estudiante con un Curso y un Pago representa una operación financiera asociada a un estudiante. Estas relaciones fueron modeladas mediante composición y asociación, evitando introducir subclases cuya única finalidad fuera satisfacer artificialmente una relación de herencia.

Esta decisión sigue el principio de favorecer la composición frente a la herencia cuando no existe una relación semántica clara de especialización. De esta manera, se mantiene un modelo de dominio más sencillo y se evita acoplar innecesariamente las entidades mediante jerarquías de clases.

No obstante, se reconoce que la presente entrega académica contempla la demostración explícita de los conceptos de herencia y clases abstractas. Por ello, la ausencia de una jerarquía de herencia constituye una limitación de la implementación actual respecto de dicho requisito de evaluación y queda identificada como un punto susceptible de ampliación en una siguiente iteración.

Clases abstractas

En la implementación actual no se incorporaron clases abstractas debido a que el modelo de dominio no presenta, en su estado actual, una jerarquía de especialización que requiera una clase base no instanciable. Las entidades principales fueron modeladas como conceptos concretos del dominio y las diferencias de comportamiento asociadas a los usuarios se gestionan mediante la relación con Rol y mediante enumeraciones de estado.

La introducción de una clase abstracta sin una especialización concreta habría añadido una capa de abstracción artificial al modelo. Por esta razón, se priorizó mantener las entidades directamente instanciables y utilizar asociación, composición e interfaces donde resultan más apropiadas.

No obstante, dado que la actividad académica contempla explícitamente la demostración del concepto de clases abstractas, este aspecto queda identificado como una oportunidad de ampliación de la solución mediante una jerarquía de especialización que responda a una necesidad real del dominio.

COLECCIONES Y ESTRUCTURAS DE DATOS

El proyecto utiliza colecciones Java de forma coherente con varios casos de uso. MatriculaService administra una colección de matrículas en memoria; CursoRepositoryImpl y PeriodoAcademicoRepositoryImpl utilizan ArrayList; PagoRepository también utiliza una colección para sus registros.

Clase	Colección	Uso
MatriculaService	ArrayList<Matricula>	Almacenar y consultar matrículas durante la ejecución.
CursoRepositoryImpl	List<Curso> / ArrayList	Gestionar cursos en memoria.
PeriodoAcademicoRepositoryImpl	List<PeriodoAcademico> / ArrayList	Gestionar períodos en memoria.
PagoRepository	ArrayList<Pago>	Registrar y consultar pagos en memoria.

La elección de List/ArrayList es apropiada cuando se requiere mantener una secuencia de objetos y recorrerlos para búsquedas o filtros. Cuando la persistencia .dat quede integrada, estas colecciones pueden funcionar como representación de los registros cargados desde archivo.

PERSISTENCIA MEDIANTE ARCHIVOS .DAT

La infraestructura de persistencia ya existe mediante DataFileManager y PersistenciaException. DataFileManager utiliza serialización Java con ObjectOutputStream para guardar objetos y ObjectInputStream para cargarlos.

```
guardar(String ruta, Object datos)
  → ObjectOutputStream.writeObject(datos)

cargar(String ruta)
  → ObjectInputStream.readObject()
```

El punto crítico de la Entrega 2 es que la infraestructura no equivale todavía a persistencia funcional de todo el sistema. En CursoRepositoryImpl y PeriodoAcademicoRepositoryImpl se verificó que los

datos permanecen en ArrayList en memoria y que actualizar() devuelve el objeto recibido sin persistirlo en archivo. Por ello, el trabajo pendiente es conectar los repositories con DataFileManager, cargar los .dat al iniciar y guardar después de operaciones de alta/modificación/eliminación.

Flujo requerido para completar la persistencia

1. Definir una ruta estable por agregado, por ejemplo data/usuarios.dat, data/cursos.dat, data/periodos.dat y data/pagos.dat.
2. Cargar la colección al construir o inicializar cada repository.
3. Si el archivo no existe, inicializar una colección vacía y crear el archivo en la primera escritura.
4. Persistir después de registrar, modificar, activar/desactivar o eliminar, según corresponda.
5. Agregar PersistenciaTest para verificar guardar, cargar y supervivencia de datos entre ejecuciones.

CONCURRENCIA

En la versión actual del sistema no se ha incorporado todavía un mecanismo de ejecución concurrente mediante hilos. Durante el análisis se identificaron procesos que potencialmente pueden beneficiarse de la ejecución concurrente, particularmente aquellos relacionados con la generación de reportes y procesamiento de información académica y financiera. Sin embargo, la implementación actual mantiene estos procesos en ejecución secuencial, por lo que no se considera que el proyecto cumpla todavía con el requisito de concurrencia establecido para este avance.

Como oportunidad de ampliación, se plantea incorporar concurrencia en procesos independientes que puedan ejecutarse simultáneamente sin comprometer la integridad de los datos. Por ejemplo, la generación de diferentes componentes de un reporte podría distribuirse entre tareas concurrentes, mientras el proceso principal coordina y posteriormente consolida los resultados. Esta implementación deberá considerar mecanismos de sincronización cuando existan recursos compartidos.

Caso de uso propuesto

Un caso coherente con el sistema es la generación de reportes de indicadores académicos y financieros en paralelo. Un proceso puede calcular el indicador de rendimiento académico mientras otro calcula el indicador de mora; posteriormente un coordinador de reportes combina ambos resultados. Esta concurrencia tiene relación directa con el dominio y evita crear un hilo artificial sin utilidad.

```
ExecutorService executor = Executors.newFixedThreadPool(2);
Future<IndicadorAcademico> academico = executor.submit(() -> reporteAcademico.generar());
Future<IndicadorMora> mora = executor.submit(() -> reporteMora.generar());
```

La implementación definitiva debe acompañarse de una prueba que demuestre el resultado de ambas tareas y el correcto cierre del executor.

PRUEBAS Y CALIDAD

La rama develop contiene pruebas JUnit para EstadoCuentaService, EvaluacionService, PagoService, ReporteService y UsuarioService. Estas pruebas evidencian una intención de validar servicios y reglas de negocio antes de la integración final.

Prueba	Objetivo observado
UsuarioServiceTest	Validar operaciones del contrato de UsuarioService mediante una implementación anónima.
PagoServiceTest	Registrar pagos y validar aprobación/rechazo.
EvaluacionServiceTest	Registrar entregas, calificarlas y consultar estado.
EstadoCuentaServiceTest	Consultar estados de cuenta y condición de mora.
ReporteServiceTest	Generar indicadores y salida de reportes.

INTEGRACIÓN Y ESTADO FUNCIONAL

Módulo / funcionalidad	Código en main	Integración	Estado funcional	Observación
Autenticación / sesión	Usuario, Sesion, Rol, AutenticacionService, LoginController	Parcial	🟡 Parcial	Existen entidades, servicio e interfaz, pero no se evidencia una implementación completa del flujo
Gestión de usuarios	UsuarioService, UsuarioController, UsuarioRepository	Parcial	🟡 Parcial	Modelo y capas principales presentes; falta comprobar implementación concreta completa
Períodos académicos	PeriodoAcademico, PeriodoAcademicoService, PeriodoAcademicoRepository, PeriodoAcademicoRepositoryImpl, AcademicoController	Sí	🟢 Funcional	Es uno de los módulos más completos

Gestión de cursos	Curso, CursoService, CursoRepository, CursoRepositoryImpl, AcademicoController	Sí	🟢 Funcional	Registro, asignación de instructor, activación/desactivación y consulta están contemplados
Matrículas	Matricula, MatriculaService	Parcial	🟡 Parcial	MatriculaService funciona en memoria, pero AcademicoController.matricularEstudiante() sigue lanzando UnsupportedOperationException
Pagos	Pago, PagoService, PagoRepository, FinancieroController	Sí/Parcial	🟡 Parcial	Existe el dominio y servicio, pero no hay evidencia de integración completa con persistencia
Facturación	Factura, FacturaService, FacturaRepository, FinancieroController	Sí/Parcial	🟡 Parcial	El módulo está presente en main, pero requiere validación de flujo completo
Estado de cuenta	EstadoCuenta, EstadoCuentaService	Sí	🟡 Parcial	Existe servicio y pruebas, pero no se observa una integración completa con todo el flujo financiero
Indicadores / reportes	IndicadorMora, ReporteService	Sí	🟡 Parcial	Hay generación de indicadores y rendimiento, pero es todavía básica
Persistencia .dat	DataFileManager, PersistenciaException	Infraestructura	🟡 Parcial	La infraestructura para guardar/cargar .dat existe, pero los repositorios

				principales siguen trabajando en memoria
Polimorfismo	Interfaces de servicios/repositorios	Parcial	🟡 Parcial	Hay abstracciones mediante interfaces, pero falta evidencia suficiente de múltiples implementaciones concretas en producción
Arquitectura por capas	Controller → Service → Repository/Entity	Sí	🟢 Implementada	La separación está claramente representada en el código
Colecciones	ArrayList, List, etc.	Sí	🟢 Implementada	Se utilizan principalmente en servicios y repositorios
Pruebas unitarias	varios *Test	Sí	🟢 Parcialmente implementada	Hay una cantidad importante de pruebas en main, aunque no cubren todo el sistema

CONTROL DE VERSIONES Y GIT

Evidencia	Descripción
Ramas	Se identifican ramas de funcionalidad como feature/usuarios-auth, feature/persistencia, feature/matriculas, feature/material-actividades, feature/reportes y otras.
Pull Requests	Los cambios de funcionalidades se integran mediante merges/PRs hacia la rama de desarrollo.
Commits individuales	El historial contiene commits asociados a funcionalidades, correcciones y pruebas.
Integración	develop concentra la integración del trabajo de las distintas ramas.

CONCLUSIONES

La Entrega 2 representa una evolución desde una especificación funcional hacia una implementación Java estructurada. El repositorio develop ya contiene entidades, enumeraciones, controladores, servicios, repositorios, infraestructura de persistencia y pruebas, lo que permite evidenciar la aplicación de varios principios de programación orientada a objetos y arquitectura por capas.

El encapsulamiento y el uso de colecciones están presentes de forma verificable. También existe una separación clara entre lógica de negocio y acceso a datos mediante servicios y repositories. La infraestructura DataFileManager constituye la base para el requisito de persistencia local, pero debe conectarse efectivamente con los repositories para que el requisito pueda considerarse completado.

Los principales puntos que requieren trabajo antes del cierre son herencia, clases abstractas, polimorfismo productivo, concurrencia y persistencia .dat integrada. Asimismo, existen operaciones del controlador académico y el módulo de facturación que deben completarse si se desea presentar el sistema como una integración funcional de todas las historias de usuario.

La estrategia recomendada para el cierre es priorizar los criterios con mayor peso en la rúbrica y que actualmente no cuentan con evidencia suficiente, sin introducir elementos artificiales que no correspondan al dominio del sistema.

BIBLIOGRAFÍA

- Pressman, R. S. (2010). Ingeniería del software: un enfoque práctico (7ma ed.). McGraw-Hill.
- Schwaber, K., & Sutherland, J. (2020). La guía de Scrum. Scrum.org.
- Sommerville, I. (2011). Ingeniería de software (9na ed.). Addison-Wesley.
- Oracle. Java Platform, Standard Edition Documentation. Referencia técnica para colecciones, serialización, concurrencia y lenguaje Java.

ANEXO A. REFERENCIAS TÉCNICAS DEL REPOSITORIO

Repositorio: 2602-Progra-II/prograii-sistema-gestion-academia

Rama revisada: main. **HEAD:** 06d2ef1136331f49235876d25a2bca11f473ad47

Ruta	Contenido relevante
pom.xml	Configuración de Maven, versión de Java, dependencias y configuración de pruebas.
src/main/java/sv/gestionacademica/entity/	Entidades del dominio: usuarios, roles, períodos, cursos, matrículas, pagos, facturación, evaluaciones y demás componentes del modelo.
src/main/java/sv/gestionacademica/enums/	Enumeraciones utilizadas para representar estados y tipos del dominio.
src/main/java/sv/gestionacademica/controller/	Controladores encargados de coordinar las operaciones de los módulos del sistema.
src/main/java/sv/gestionacademica/service/	Servicios y reglas de negocio de los diferentes módulos académicos y financieros.
src/main/java/sv/gestionacademica/repository/	Interfaces que definen los contratos de acceso y gestión de datos.
src/main/java/sv/gestionacademica/persistence/	Componentes de persistencia mediante archivos .dat, incluyendo DataFileManager y las excepciones asociadas.
src/main/java/sv/gestionacademica/exception/	Excepciones propias del sistema, incluyendo PersistenciaException.
src/test/java/sv/gestionacademica/	Pruebas automatizadas existentes para los servicios implementados.
docs/uml/	Diagramas relacionados con el modelo de clases, modelo relacional y arquitectura del sistema.
docs/documentos/	Documentación de las entregas académicas y distribución de responsabilidades del equipo.

Enlace a video explicativo del repositorio: https://drive.google.com/file/d/10PQ_U9wnxJ_wQa_EhV_Q9eDmRiah89yv/view?usp=sharing